

# Maskinspråkskoder

Referensdokument för Assembler-Simulatorn · AI 1 · Erik Dahlbergsgymnasiet

## Datorns komponenter

Simulatorn modellerar en enkel 8-bitars dator med tre huvuddelar: minne (RAM), en räkneenhet (ALU) och en kontrollenhet. Program skrivs direkt i minnet som hexadecimala koder. Varje instruktion tar exakt två minnesceller (4 hex-siffror).

| Komponent                 | Storlek | Antal       | Notation      |
|---------------------------|---------|-------------|---------------|
| Register                  | 8 bit   | 16 st (0-F) | R0 ... RF     |
| Minne (RAM)               | 8 bit   | 256 celler  | 00 ... FF     |
| Programräknare (PC)       | 8 bit   | 1 st        | hex-adress    |
| Instruktionsregister (IR) | 16 bit  | 1 st        | 4 hex-siffror |

## Instruktionstabell

En instruktion består av en op-kod (första hex-siffran) följt av operanden (de tre resterande siffrorna). R, S, T betecknar registernummer (0-F). XY betecknar en 8-bitars minnesadress.

| Op | Operand | Funktion | Beskrivning                                                                      |
|----|---------|----------|----------------------------------------------------------------------------------|
| 1  | RXY     | LOAD     | Ladda (LOAD) register R med bitmönstret från minnescell XY.                      |
| 2  | RXY     | LOAD     | Ladda (LOAD) register R med det omedelbara värdet XY.                            |
| 3  | RXY     | STORE    | Spara (STORE) innehållet i register R till minnescell XY.                        |
| 4  | ØRS     | MOVE     | Flytta (MOVE) bitmönstret i register R till register S.                          |
| 5  | RST     | ADD      | Addera (ADD) register S och T. Spara resultatet i R. Tolkas som 2-komplementtal. |
| 6  | RST     | FADD     | Addera (ADD) register S och T som 8-bitars flyttal (seeemmm). Spara i R.         |
| 7  | RST     | OR       | Bitvis OR på register S och T. Spara resultatet i register R.                    |
| 8  | RST     | AND      | Bitvis AND på register S och T. Spara resultatet i register R.                   |
| 9  | RST     | XOR      | Bitvis XOR på register S och T. Spara resultatet i register R.                   |
| A  | RØX     | ROTATE   | Rotera (ROTATE) bitmönstret i register R cirkulärt åt höger X steg.              |
| B  | RXY     | JUMP     | Hoppa (JUMP) till adress XY om register R är lika med register 0.                |
| C  | ØØØ     | HALT     | Stoppa (HALT) programkörningen.                                                  |
| D  | RØX     | SHL      | Skifta register R åt vänster X steg. Bit 7 (MSB) bevaras. Nya bitar = 0.         |

| Op | Operand | Funktion | Beskrivning                                                                 |
|----|---------|----------|-----------------------------------------------------------------------------|
| E  | R0X     | SHR      | Skifta register R åt höger X steg. Bit 7 (MSB) bevaras (aritmetiskt skift). |

Flyttalsformat (op-kod 6)

Op-kod 6 använder ett 8-bitars flyttalsformat med layouten seeemmmm där s = teckenbit, eee = exponent (excess-4) och mmmm = mantissa.



Värde =  $(-1)^s \times 0.mmmm_2 \times 2^{exp-4}$  · Exponent lagras med excess-4: värde 4 → eee = 100

Exempel 1 — Ladda och avsluta

Skriv ett program som laddar register 2 med det värde som finns på minnesadressen A0, och avslutar sedan programmet. Placera programmet från adress 00.

| Minne (hex) |                                    |
|-------------|------------------------------------|
| 00-01       | 12 A0 — LOAD R2 ← MEM[A0]          |
| 02-03       | C0 00 — HALT — avsluta programmet  |
| A0          | FF — Indata: värdet som ska laddas |

Efter körning: R2 = FF · PC = 04 · IR = C000

Exempel 2 — Addition (2-komplement)

Addera talen på adress B0 och B1. Spara resultatet på adress B2.

| Minne (hex) |                                             |
|-------------|---------------------------------------------|
| 00-01       | 10 B0 — LOAD R0 ← MEM[B0] (första talet)    |
| 02-03       | 11 B1 — LOAD R1 ← MEM[B1] (andra talet)     |
| 04-05       | 52 01 — ADD R2 ← R0 + R1 (2-komplement)     |
| 06-07       | 32 B2 — STORE MEM[B2] ← R2 (spara resultat) |
| 08-09       | C0 00 — HALT                                |

## Att skriva och köra ett program

### 1. Skriv koden i minnet

Klicka på en minnescell och skriv in ett hexadecimalt värde (00–FF). Varje instruktion tar två celler i följd.

### 2. Ställ in Programräknaren (PC)

PC ska peka på den cell där programmet börjar — oftast 00.

### 3. Kör programmet

Steg (F8) — kör en instruktion i taget. Bra för felsökning. · Kör (F9) — kör hela programmet till HALT.

### 4. Spara och ladda

Spara programmet som en .sim-fil via knappen Spara. Öppna en sparad fil med Öppna.

## Notation

| Symbol  | Betydelse                             | Exempel        |
|---------|---------------------------------------|----------------|
| R, S, T | Registernummer 0–F                    | R2, S0, TA     |
| XY      | 8-bitars minnesadress (2 hex-siffror) | A0, FF, 00     |
| 0       | Oanvänd/nollnibble i operanden        | 40RS → 4 0 R S |
| MEM[XY] | Innehållet i minnescell XY            | MEM[A0] = FF   |